DiffInk 应用中出现的问题

在我们自己的数据集使用DiffInk的时候，无法进行正常的输出（可见§3），只有训练分布内的输入能成功，且VAE部分未出现问题，想请作者帮判断，是模型的哪个部分出现问题？应该如何进行解决？

文档结构：

- §1 讲 **gfont → DiffInk 五元组输入格式**的转换细节（纯格式说明）。
- §2 对比 **三种 style reference 输入来源**（gfont 转、真实手写图 → InkSight、val.h5 直接取样）的数据统计。
- §3 对比三种来源在 DiffInk 上的 **完整推理输出效果**。
- §4 **VAE-only 重建诊断**：单独跑 VAE encode→decode，隔离 VAE 和 diffusion 两部分。
- §5 汇总想请原作者确认的问题。

---

1. gfont → DiffInk 输入格式的转换

入口函数：

```
gfont_to_diffink(chars, font_data, char_gap=30.0, target_y_range=5.0)
    -> (point_seq: np.ndarray[T, 5], char_points_idx: list[int])
```

返回的 `point_seq` 和 `char_points_idx` 直接作为 DiffInk handler 的 `style_strokes`（base64 编码后）与 `char_points_idx` 字段送入。

五步流程：

1. **解析 gfont**：`.gfont` ZIP 里每个文件按 Unicode 码点命名，读二进制得到 `{char: [stroke_array, ...]}`（每笔 `[Ni, 2]` 浮点点序列）。
2. **逐字预处理**：x 平移到该字左边缘为 0；y 取反（gfont 是屏幕坐标 y 向下 → CASIA 数学坐标 y 向上，由 `visual.py` 渲染器 `y_scaled = (y_max - y) * scale` 反证）。此阶段不缩放。
3. **五元组 `[x, y, is_next, is_new_stroke, is_new_char]`**：笔画内部点 `(1,0,0)`、笔画末（非该字最后一笔）`(0,1,0)`、字符结束标记 `(0,0,1)`。
4. **水平拼接 + 记录边界**：每字后 `x_offset += char_w + char_gap`；`cpi[i] = len(all_points)` = 下一字第一点的索引（**右开切分点**，末字 `cpi[-1] = T`）。格式从 val.h5 真实样本反推对齐（31 字样本 `cpi=[9, 21, ..., 802]`, 总点 802, `cpi[-1]==T`），无额外"过渡点"插入。
5. **全局等比缩放**：`scale = target_y_range / y_range`（`target_y_range=5.0`）同时乘 x 和 y（**不逐字缩放**，保留字间相对尺度），y 居中到 0，最终 y ∈ [−2.5, +2.5]。

修复记录：早期版本在结束标记后多插了一个冗余"过渡点"导致 `cpi[-1] = T-1`、边界 flag 变成 `(0,0,1) → (1,0,0) → (1,0,0)`。对比 val.h5 后已修。

---

2. 三种 style reference 输入来源对比

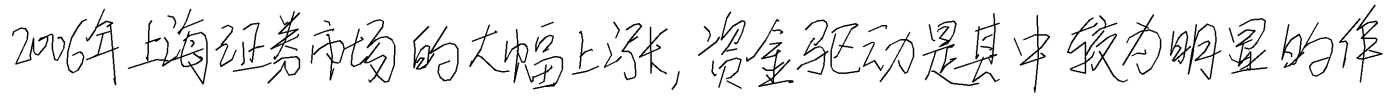
为了定位推理失败的根因，把三种来源的 style reference 放在同一套 handler 上跑，形成 **输入** → **输出** 的端到端对照。[1] 和 [2] 都按 §1 的流程对齐到 CASIA 格式（`cpi[-1]=T`、无冗余过渡点、y 翻转、y 范围 5.0）。

2.1 输入轨迹对比

[1] gfont to 5-tuple: our input (10 chars, style 5 + target 5)

[2] Real handwriting image to InkSight to 5-tuple (7 chars: ren sheng de yi xu jin huan)

[3] val.h5 CASIA sample (31 chars): training-distribution reference



#	来源	字串	特点
[1]	**gfont 转 5-tuple**（本文重点）	天地黄宇宙洪荒严寒永（10 字）	由矢量字体直接转出；字迹规整、采样密
[2]	**真实手写图 → InkSight → 5-tuple**	人生得意须尽欢（7 字）	手写照片经 Google InkSight 还原笔顺；已按 §1 流程对齐
[3]	**val.h5 直接取样**	2006 年上海证券市场…（31 字）	训练分布内，5-tuple 直接来自 h5

2.2 格式/统计全维度对比

把 [1] 和 [3] 同维度对比（InkSight 路径的 [2] 数据量少，这里先不列进表）：

维度	[3] val.h5 CASIA	[1] 我们 gfont	比例	判断
y 范围	5.44	5.00	0.92×	接近但略小
x 跨度 / 字	均值 2.54	均值 3.71	1.46×	⚠ 我们字偏宽
字间距（inter-char gap）	均值 2.85（min 0）	均值 4.18（min 1.36）	1.47×	⚠ 我们字间距偏大、无 0 gap
每字点数	均值 26，中位 28	均值 40，中位 39	1.54×	⚠ 我们采样密度显著偏高
笔画内步长	均值 0.68，中位 0.51	均值 0.50，中位 0.37	0.74×	⚠ 我们步长短（oversampling）
每字笔画数（`(0,1,0)` 计数）	~4.5 笔/字	~5.8 笔/字	1.29×	略多，gfont 字体本身笔画多
`cpi[-1] == T`	✓	✓（已修）	—	✓

主要偏差：我们的 gfont 输入在**采样密度 / 字间距 / 单字宽度**三个维度上都约是 CASIA 的 1.5×，y 范围接近但略小。

---

3. DiffInk 推理输出对比

3.1 三种 style reference 来源对比

[1] DiffInk output from gfont input: style prefix OK, target chars missing/garbled

[2] DiffInk output from InkSight input: also partial (first few chars visible, rest broken)

[3] DiffInk output from val.h5 input: full sentence regenerated correctly

#	输入来源	DiffInk 输出结果
[1]	gfont → 5-tuple	style prefix 的 5 字（天地黄宇宙）能还原；target 5 字（洪荒严寒永）缺失或几笔残迹，且位置与 style 区重叠
[2]	InkSight 还原的真实手写	类似 [1]：前几字可辨，后续糊成团
[3]	val.h5 直接输入	**完整 31 字句子正确重建**，训练分布内端到端畅通

关键观察：即使 [1] 和 [2] 都按 §1 流程对齐到 CASIA 格式（`cpi[-1]=T`、无冗余过渡点、`y_range=5.0`），**target 区仍然崩坏**；只有训练分布内的 [3] 能跑出完整可读的字。这说明：

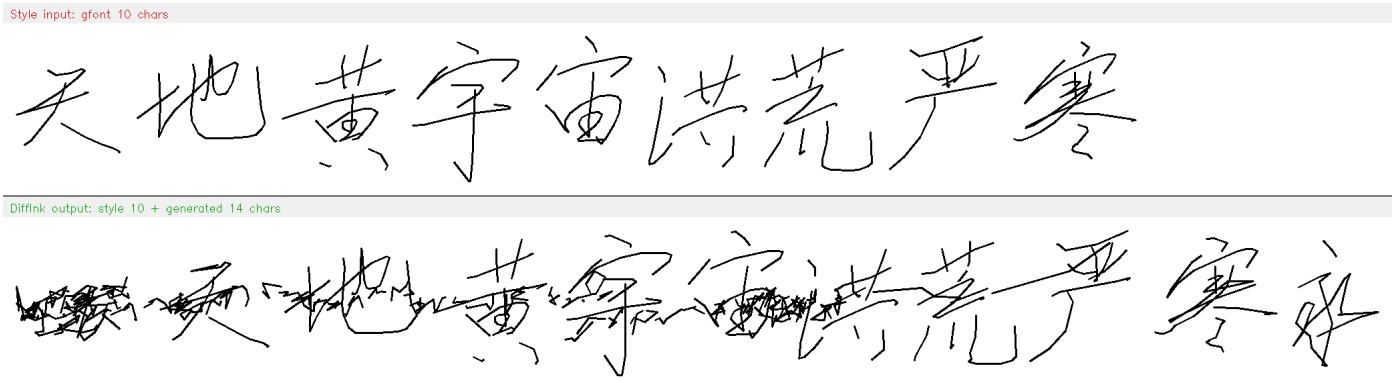
- 问题不在模型本身（val.h5 输入能正常工作）；
- 问题也不单纯在"char_points_idx 格式 bug"或"y 缩放"上——即便修完这两处，[1] 和 [2] 依旧糊；
- [1]（gfont 矢量）和 [2]（InkSight 还原）的失败模式高度相似，说明瓶颈更深——大概率是 §2.2 表里其它维度（采样密度、字间距、单字宽度）或未覆盖到的训练侧预处理环节造成的 distribution shift。

3.2 gfont 路径：长度与 style/target 组合消融

§3.1 [1] 只跑了"5 style + 5 target、target 字全在 style 集内"一种配置。为了看失败模式是否随长度/style 覆盖率变化，在**修复后的 gfont 格式**（`cpi[-1]=T`、无冗余过渡点、y-range=5）上加跑三组消融：

#	试验	style 笔迹	target 文本	target 是否与 style 重叠	结果
A	**sentence**	10 字 gfont 笔迹（天地黄宇宙洪荒严寒永）	14 字古诗"春风又绿江南岸明月何时照我还"	**完全不重叠**	prefix 10 字清晰还原；target 14 字糊成团，位置还与 style 重叠
B	**long**	5 字（天地黄宇宙）	16 字"洪荒严寒日月星辰春风秋雨山高水长"（style 集之外 14 字）	不重叠	几乎全部崩坏，只在右侧勉强认出"黄宇宙"
C	**10s10t**	10 字（天地黄宇宙洪荒严寒永）	10 字"日月星辰春风秋雨山高"	**完全不重叠**	style 区 10 字清晰可辨；target 区笔势方向对但字形糊
D	**short**	5 字（天地黄宇宙）	5 字"洪荒严寒永"	**完全重叠**（target 5 字都在 ref 的后 5 字 gfont 笔迹里）	与 §3.1 [1] 同一配置、新实例复现，结果一致：style 5 字清晰，target 5 字几乎全崩（`地` 位置只剩一团乱笔）

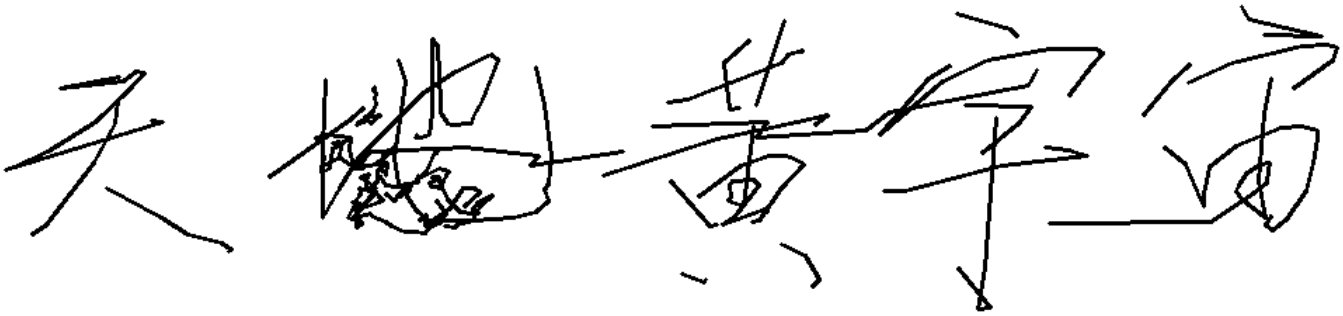
A — sentence（10 style → 14 target，style 与 target 完全不重叠）：上 = style 笔迹输入，下 = DiffInk 生成结果。



B — long（5 style → 16 target）： DiffInk 输出（style 为 §2.1 图中前 5 字 `天地黄宇宙`）。

C — 10s10t（10 style → 10 target，target 完全新字）： DiffInk 输出（前 10 字为 style prefix，后 10 字为生成 target）。

D — short（5 style → 5 target，target ⊆ style 的 gfont 轨迹）： DiffInk 输出。

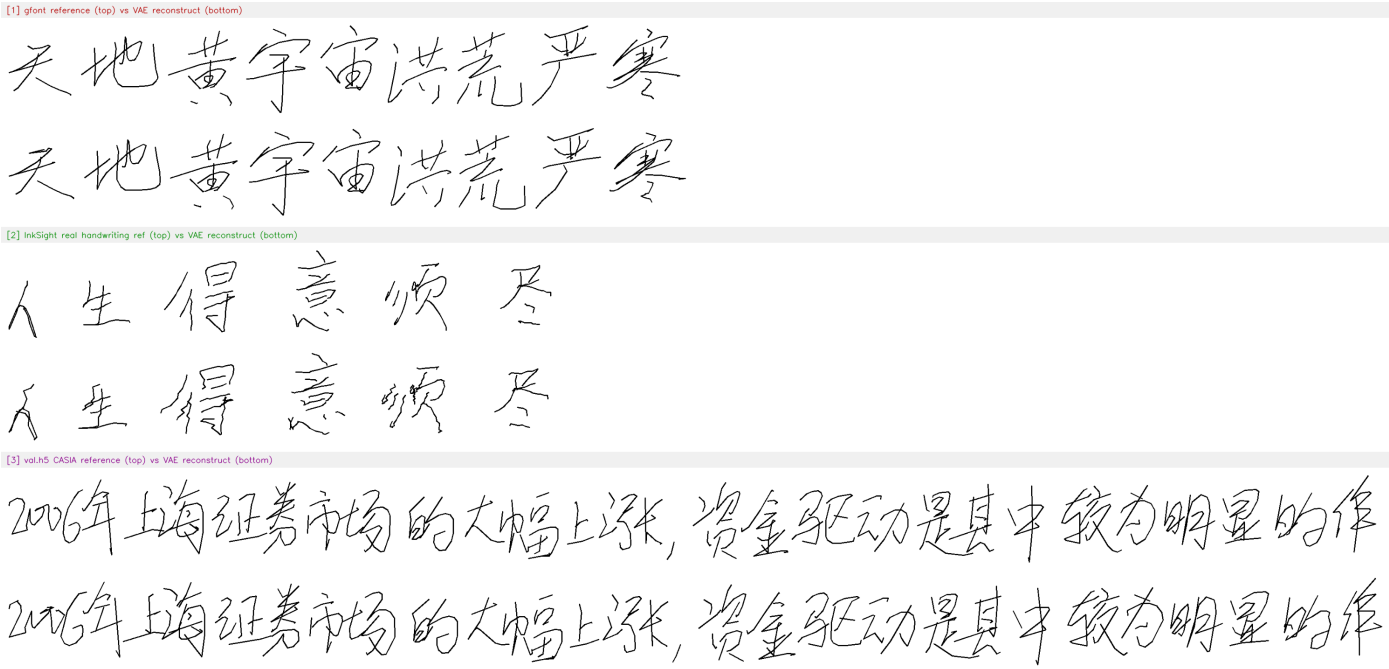


观察：

1. **style prefix 区始终能正确还原**（A/C/D 都能读出相应的 style 字、B 能读出 5 个）——说明 prefix mask 的切分没问题，VAE→DiT 对已知 latent 的 passthrough 正常。
2. **target 区在 A/B/C/D 四种配置下全部崩坏**。D（target 完全包含在 style ref 的 gfont 轨迹里，最"简单"的一档）同样糊——说明 target 崩坏**和 target 字是否在 style 集合里没关系**，不是"模型没见过这个字"的问题。
3. **10s10t (C) 对比 D（5s5t）**： 把 style 从 5 加到 10（条件信息量翻倍）后 target 并没有明显变好，只是笔势方向更像手写——说明堆更多 style 条件**不是**瓶颈。
4. **sentence (A) 对比 long (B)**：同样是不重叠 target，10 字 style 比 5 字 style 的 target 稍好但仍糊；和 §2.2 的 distribution shift 结论一致，target 区越长、DiT 越发散。

4. VAE-only 重建诊断

为了把 §3 的失败定位到 pipeline 的哪一层，单独跑一个脚本：**只做 `VAE.encode → decode → GMM sample`，不经过 DiT / diffusion / prefix mask**。



每组上半 = reference，下半 = VAE 重建：

#	输入	VAE 重建质量	latent 统计
[1]	gfont 5-tuple（本文重点）	**几乎完美重建**（天地黄宇宙洪荒严寒[永] 所有字都可辨）	mean=−0.044 std=1.208 min=−6.12 max=5.18
[2]	InkSight 还原的真实手写	**几乎完美重建**（人生得意须尽[欢] 所有字都可辨）	mean=−0.052 std=1.007 min=−5.66 max=5.71
[3]	val.h5 CASIA 样本	**几乎完美重建**（31 字完整句子）	mean=−0.093 std=1.268 min=−9.37 max=11.34

结论：VAE **不是瓶颈**。三种输入（无论训练分布内还是我们新构造的 gfont / InkSight）经 VAE 都能干净重建。这把 §3 的失败定位到 **diffusion 模型 + handler 的条件/掩码策略**上——VAE 的 latent 空间对我们的输入是完整可表达的，但 DiT 在 suffix mask 区域去噪回来的 latent 解不出可读的字。

（唯一可见的统计差异：[3] 的 latent 最大/最小绝对值（11.34 / 9.37）明显比 [1][2] 大——可能因为序列更长，但也可能意味着我们的输入使 latent 分布偏中心化，信息密度不够。）